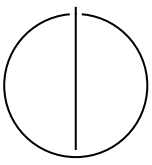
SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY — INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

Adaptation Techniques for using NAS Methods in the FL Setting

Max Coetzee



SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY — INFORMATICS

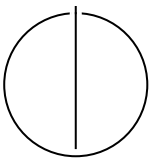
TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

Adaptation Techniques for using NAS Methods in the FL Setting

Titel

Author:	Max Coetzee
Examiner:	Supervisor
Supervisor:	Advisor
Submission Date:	Submission date



I confirm that this bachelor's thesis is my own work and I have documented all sources and material used.

Munich, Submission date Max Coetzee

Acknowledgments

Abstract

Both *Neural Architecture Search* (NAS) and *Federated Learning* (FL) have made significant progress independently in the past decade. To benefit from the advantages of NAS methods in FL, researchers have started combining them by using NAS in FL [10] [5] [14].

[TODO: overhaul]

Applying techniques from Neural Architecture Search (NAS) to Federated Learning (FL) has been fruitful (remove) in recent years. The combination was identified as a promising research (remove) direction by [12]. It has yielded methods for finding architectures that deal with the challenges imposed by the FL setting.

Research into NAS has grown rapidly [19] since it was popularized by [25]; consequently, literature on its application to FL has grown. The last survey on NAS applied to FL compared approaches of four papers [24]. Since then, we have identified approximately 50 new papers. This motivates a new systematic survey of the landscape to identify progress and gaps in the literature.

In this thesis, we propose a map of the literature landscape based on the FL challenges they address. We achieve this by systematically evaluating the literature and identifying which challenge it solves.

We refer to the FL challenges described in [25], i.e., non-IID data, limited communication, client heterogeneity, privacy of client data, and break them down into smaller subchallenges — each subchallenge being associated with a pattern in the literature. We include personalized FL [18] as an additional subchallenge that was not originally posited, but has since drawn the community's attention.

We then analyze how the subchallenges are addressed and focus on the contribution of the used NAS method towards overcoming the subchallenge. For each subchallenge, we keep track of the NAS types used (following [19], [2]) and assess whether the underexplored methods are candidates for future research.

Neural Architecture Search

Contents

1 Introduction

Deep learning has seen widespread adoption over the last decade and is being applied to an increasingly diverse set of tasks. Applying deep learning traditionally involves a team of deep learning and domain experts who tailor a neural network architecture to a specific task through a lengthy process of trial and error. To automate this process, researchers invented *Neural Architecture Search* (NAS) [7] methods. NAS methods employ diverse strategies to automatically search for a neural network architecture for a given deep learning task within an architecture search space.

Independently, but in parallel to NAS, researchers developed a machine learning approach called *Federated Learning* (FL) [15], which enables training an ML model on data distributed across a set of clients without sharing their data. The model is distributed from the server to the clients, which train the model on their local data. The clients then send the model weight updates to the server, which aggregates them and redistributes them to the clients. This process is repeated for several rounds. FL enhances the privacy of client data and enables training even when clients' data is too large to transfer effectively to a central training site or cannot be shared due to regulatory or privacy constraints.

When practitioners[1] make use of deep learning in FL, they typically use a predefined architecture. However, clients' data distributions and hardware capabilities are highly heterogeneous in FL. Since practitioners cannot directly observe these client characteristics [10], it is hard to choose a single architecture that generalises well across all clients and that meets inference latency constraints on every client. Using NAS methods in FL provides an alternative: the server can coordinate an architecture search across clients, guided by performance feedback from candidate architectures evaluated directly on clients. Thus, an architecture can be found that is tailored to the clients' characteristics.

Despite the benefit, using NAS in FL is not straightforward. Most NAS methods were developed for a centralised setting and rely on assumptions that often do not hold for FL. These assumption discrepancies create several *challenges* for using NAS in FL. For example, in a centralised setting, practitioners can expect a NAS method to run on worker nodes connected via low-latency, high-bandwidth links. However, in FL, a NAS method may run on clients connected via an unreliable, high-latency, low-bandwidth network. This creates a challenge when transferring the weights of

[1] In this thesis, *practitioners* refers to both users and developers of NAS methods for FL.

large candidate models between clients and the server to coordinate an architecture search. Practitioners need specialised techniques to overcome such challenges, such as only transferring encodings of candidate architectures to clients [17].

Many techniques for overcoming such challenges already exist, but practitioners need to comb an increasingly large body of NAS-in-FL literature for suitable techniques (i.e., techniques that overcome the challenges relevant to their targeted FL system characteristics). The relevance of a challenge depends on the extent to which the targeted FL system characteristics violate centralised NAS assumptions. FL system characteristics include the average hardware capabilities of clients, the average network latency of clients, the number of participating clients, etc. Additionally, practitioners need to weigh which techniques to use to overcome their relevant set of challenges, since a technique for overcoming one challenge may inadvertently make it harder to overcome another.

However, finding suitable techniques is difficult. Prior surveys of the NAS-in-FL literature categorise NAS-in-FL methods as a whole rather than analyse the individual techniques they use. [24] categorises NAS-in-FL methods into offline versus online architecture search and single-objective versus multi-objective methods. [13] gives a high-level overview of the NAS-in-FL landscape at the time of its publication as part of a larger survey into combining NAS and Hyperparameter Optimisation. [9] investigates how multi-objective optimisation can be integrated into FL in general and includes only parts that discuss how this is done specifically for NAS-in-FL methods. Moreover, all of the surveys analyse only a small fraction of the entire NAS-in-FL literature. As a result, knowledge on challenges when using NAS in FL and the techniques for overcoming those challenges remains fragmented across several studies. A more sophisticated understanding of the challenges and remedying techniques for using NAS in FL is needed to allow practitioners to find techniques that could overcome the set of challenges relevant to their targeted set of FL system characteristics. Therefore, we ask the following research question:

What challenges arise when using NAS in FL, and what techniques overcome these challenges in the literature?

To tackle our research question, we conduct an extensive systematic literature review. We collect an initial set of relevant literature with a Scopus [6] search and extend it with a snowball search. We first use open coding to identify, from the literature, assumption discrepancies and resulting challenges, when using NAS in FL. Next, we extract FL system characteristics and use axial coding to code the influence of characteristics on the relevance of challenges. Then, we extract techniques through open coding and iteratively refine them to synthesise a coherent set of mutually exclusive and collectively

exhaustive techniques. Finally, we use axial coding to map techniques onto challenges they overcome.

Our review aims to synthesise techniques for using NAS in FL from the literature to make finding suitable techniques easy in practice. We make the following three contributions. First, by providing a consolidated overview of the challenges of using NAS in FL, practitioners can grasp which issues they can expect to encounter at a glance. Second, by describing how FL system characteristics influence the relevance of challenges, we enable practitioners to focus on the challenges relevant to them. Third, by extracting techniques for using NAS in FL and highlighting the challenges they address, we enable practitioners to compare existing techniques for using NAS in FL for their specific set of challenges.

In Chapter 2, we cover the background required for this thesis and related work. In Chapter 3, we describe the method with which we perform our literature review and give an overview of the included literature. In Chapter 4, we highlight the challenges with FedNAS and how they arise from different FL settings. In Chapter 6, we describe the adaptation techniques we synthesised and how they overcome challenges. In Chapter 7, we conduct a discussion about our work. Chapter 8 contains our conclusion.

2 Background and Related Work

NAS is contained entirely in the 5th step of the FL pipeline as described in [12].

2.1 Neural Architecture Search

When practitioners make use of Deep Learning, they usually design the architecture of a neural network manually before training it. This design process usually consists of tedious rounds of trial and error that is guided by heuristics and practitioners' experience. As a way to save time and make this process more reproducible, practitioners have attempted to automate it, which has culminated in a wide variety of methods. As a way to speed up and improve this process, practitioners invented methods for automatically designing the architecture of a neural network by searching for architectures that are suited to being trained well for a certain task

2.2 Federated Learning

2.3 Federated Neural Architecture Search

3 Method

[TODO: overhaul to include method for identifying challenges and FL settings] - targeted FL setting can be stated directly, indirectly or must be inferred from experimental setup

We use various techniques from [3] and lean on parts of the methodology of [11] to perform a systematic literature review.

1. **Literature Selection:** We follow the guidelines and flow diagrams provided by PRISMA 2020 [16] for inclusion and exclusion of papers and perform forward and backwards citation searching. We identify 59 papers that present FedNAS methods. Each paper contains one or more FedNAS methods.

2. **Unrefined Adaptation Technique Extraction:** Once the set of included papers is fixed, we perform open coding on each FedNAS method to extract unrefined adaptation techniques. Any modification to a NAS method that is explicitly motivated by the federated setting is treated coded as one unrefined adaptation technique.

3. **Adaptation Techniques Coneceptualization:** We iteratively refine and merge unrefined adapatation techniques in an axial coding step to obtain a coherent set of adaptation techniques that are mutually exclusive and collectively exhaustive. Unrefined adaptation techniques with conceptually highly-similar mechanisms are merged into a single representative adaptation technique.

4. **Categorise Adaptation Techniques:** After merging, in a second axial coding step, we cluster adaptation techniques based on a) the FedNAS challenges they address and b) the conceptual similarity of their mechanisms. As a result, we obtain a taxonomy of adaptation techniques.

5. **Discuss FedNAS Challenges for Adaptation Techniques:** We discuss how each adaptation technique works towards, against, or does not affect overcoming each of FedNAS challenges.

6. **Produce Table Overviews:** Finally, we create two tables that practicioners can use to make decisions about FedNAS methods for their use case.

The first table contains a coded vector of effects over the FedNAS challenges for each *FedNAS method*. The effects per FedNAS method are the result of the effects of its adaptation techniques in aggregate. Practicioners can use this table to decide which FedNAS method suits their use case or if their use case would benefit from a new FedNAS method.

The second table table contains a coded vector of effects over the FedNAS challenges for each *adaptation technique* based on the prior discussion. If practicioners need to create new FedNAS methods, they can use this table to choose existing adaptation techniques relevant to the set of FedNAS challenges they need to address for their use case.

3.1 Reviewed Literature

add some overview and statistics about the literature

4 FedNAS Challenges

Assumption in Centralised Setting	Reality in FL Setting	Resulting FedNAS Challenge Description	FedNAS Challenge Name
Worker nodes' hardware is homogeneous.	Clients' hardware varies significantly. More pronounced in the cross-device setting than the cross-silo setting.	FedNAS methods need to be heterogeneity-aware; otherwise, the search will be delayed by low-end devices or leave high-end devices waiting in idle most of the time.	Hardware Heterogeneity
Worker nodes communicate over high-bandwidth, low-latency links.	Clients typically communicate with the central server over high-latency and low-bandwidth connections.	FedNAS must be communication-efficient; otherwise, exchanging model weights and architecture parameters becomes a bottleneck during the search process.	Limited Networking Capabilities
Worker nodes are high-end machines with powerful CPUs, GPUs, and large amounts of RAM.	Clients are edge devices with few computational resources.	The computational burden placed on clients by a NAS method needs to be adjusted such that the architecture search takes place within an acceptable time frame. This can involve splitting up computational work, usually done on single worker nodes, amongst multiple clients, making the implementation complex.	Limited Computational Resources
Training data is gathered at a central location.	Training data is distributed unevenly across clients	Since some clients have more data than others, FedNAS methods must ensure that these clients are not overrepresented in the searched architecture.	Unbalanced Client Data

The training data is drawn from the same distribution.	Each client draws training data from their own distribution.	FedNAS methods need to be robust with respect to non-i.i.d. data, which makes evaluating and ranking candidate architectures noisier than in the centralised setting.	Non-I.I.D. Training Data
Worker nodes are equally available.	Some clients are more frequently available than others.	FedNAS methods must prevent the search from being dominated by the most frequently available clients; otherwise, the resulting architecture will be biased towards them.	Variable Client Availability
All worker nodes participate in each iteration.	Only a subset of clients participates in each iteration of the search.	FedNAS methods must ensure that a realistic sample of the client population is represented in the architecture search and address high-variance performance estimates, as candidate architectures are evaluated on changing subsets of clients.	Client Participation
Worker nodes are inside the same trust domain.	All participating parties (i.e. the central server and clients) can consider another potentially malicious.	FedNAS methods must address attacks from participating parties, such as architecture parameter poisoning during the search process.	Security
Worker nodes consistently participate in the search process.	Clients can drop out of a communication round at any time, and completion times of epochs vary.	FedNAS methods need to handle client dropouts and stragglers, since interrupted or delayed evaluations of candidate architectures can slow down or destabilise the search process.	Client Reliability

Model accuracy is more important than model resource consumption in selecting architectures.	Model accuracy needs to be traded for model resource consumption in selecting architectures.	Models trained in FL tend to be deployed on the same resource-constrained clients they are trained on. Therefore FedNAS methods need to optimise the architecture of the trained model w.r.t. resource constraints.	Model Resource Constraints

Table 4.1: Discrepancies between NAS in the centralised setting and the FL setting and the resulting FedNAS challenges. *Worker nodes* perform the architecture search in the centralised setting, whereas a *server* and *clients* perform the search in the FL setting. The challenges are based to some extent on previously identified challenges in the FL setting in general [15] [12] [4] [1].

5 Using NAS in FL

We identified five overarching classes of challenges present in the NAS-in-FL literature: Client Resource Constraints, Heterogeneity, Client Unreliability, Security and Architecture Search Coordination. The severity and applicability of challenges depends on the FL system characteristics as well as the the NAS-in-FL archetype, which we will explain in the following.

Each class of challenges contains several challenges and for most challenges there are several proposed solutions in the literature. In the following we present the challenges and their solutions. The challenges and appropriate solutions depend on the FL system characteristics as well as the archetype of the NAS-in-FL method.

5.1 Client Resource Constraints

Even in a centralised setting, making NAS work is difficult due to the expense of searching and evaluating many neural network architectures over many iterations. FL system setups typically feature even less computational resources, exacerbating the difficulty with making NAS work. This is one of the core issues of using NAS in FL: making it work despite the lack of resources of participating clients.

5.1.1 Computational Constraints

This challenge is present across all NAS-in-FL archetypes in a cross-device FL setting.

NAS increases the amount of computation compared to training predefined architectures, because clients must train or evaluate candidate architectures before one can be selected [21, 8]. This additional expense makes cross-device FL particularily challenging, where clients may be mobile phones or IoT[1] devices whose limited computing capacity must also support their primary workload [23, 8]. [21] reports architecture search times in the range of multiple hours to reach a $> 70\%$ test accuracy on CIFAR-10 with a testbed of 30 NVIDIA Jetson TX2 devices. Though technically edge devices, they are more performant than an average mobile phone, therefore longer search completion

[1] Internet of Things

times can be expected in practice and practitioners must go to great lengths to optimise the search process to make it take a feasible amount of time.

Computational constraints are less severe in cross-silo FL, where clients may be GPU-equipped servers operated by participating organisations [23, 10].

Training One Supernet Path at a Time

Single-path training updates one sampled path through the supernet at each step instead of evaluating every candidate operation, reducing the computation to that of training one network [8].

Pruning the Search Space during Training

Progressive pruning removes the weakest operation from each edge at fixed intervals, reducing the supernet computation required in later search rounds [21].

Limiting Candidate Fine-Tuning

Dynamic round budgets shorten early candidate evaluations, while early elimination stops weak candidates after a few rounds so that only the best receives extended training [23].

Reusing Supernet Weights

Weight inheritance evaluates candidates with trained supernet weights instead of retraining them from scratch, reducing the reported retraining cost from minutes to seconds [20].

Predicting Candidate Performance

A performance predictor learns from a small evaluated sample and estimates the remaining candidates without running each one on the client [22].

5.1.2 Challenges caused by Communication Constraints

5.1.3 Challenges caused by Memory Constraints

5.2 Heterogeneity

In FL, clients are heterogenous in many aspects. This makes using NAS particularily difficult.

5.2.1 Client Hardware Heterogeneity

5.2.2 Client Data Heterogeneity

- different data distributions - unbalanced data

5.2.3 Client Network Heterogeneity

5.3

- how can clients be different?

6 Adaptation Techniques

6.1

7 Discussion

- using NAS in FL still lags behind regular FL in several key areas: scale of training fleets, etc.

8 Conclusion

Abbreviations

List of Figures

List of Tables

Bibliography

[1] M. Arbaoui, M.-A. Brahmia, A. Rahmoun, and M. Zghal. "Federated learning survey: A multi-level taxonomy of aggregation techniques, experimental insights, and future frontiers." In: *ACM Trans. Intell. Syst. Technol.* 15.6 (Nov. 2024). ISSN: 2157-6904. DOI: 10.1145/3678182.

[2] S. S. P. Avval, N. D. Eskue, R. M. Groves, and V. Yaghoubi. "Systematic review on neural architecture search." In: *Artificial Intelligence Review* 58.3 (Jan. 6, 2025), p. 73. ISSN: 1573-7462. DOI: 10.1007/s10462-024-11058-w.

[3] J. Corbin and A. Strauss. *Basics of qualitative research*. Core textbook v. 14. SAGE Publications, 2015. ISBN: 9781412997461.

[4] K. Daly, H. Eichner, P. Kairouz, H. B. McMahan, D. Ramage, and Z. Xu. "Federated learning in practice: Reflections and projections." In: *2024 IEEE 6th international conference on trust, privacy and security in intelligent systems, and applications (TPS-ISA)*. 2024, pp. 148–156. DOI: 10.1109/TPS-ISA62245.2024.00026.

[5] L. Dudziak, S. Laskaridis, and J. Fernandez-Marques. *FedorAS: Federated architecture search under system heterogeneity*. 2022. arXiv: 2206.11239 [cs.LG].

[6] Elsevier. *Scopus*. Abstract and citation database. Accessed 2026-01-17. 2026.

[7] T. Elsken, J. H. Metzen, and F. Hutter. "Neural architecture search: A survey." In: *Journal of Machine Learning Research* 20.55 (2019), pp. 1–21.

[8] A. Garg, A. K. Saha, and D. Dutta. *Direct federated neural architecture search*. 2020. arXiv: 2010.06223 [cs.LG].

[9] M. Hartmann, G. Danoy, and P. Bouvry. *Multi-objective methods in Federated Learning: A survey and taxonomy*. 2025. arXiv: 2502.03108 [cs.LG].

[10] C. He, M. Annavaram, and S. Avestimehr. *Towards Non-I.I.D. and invisible data with FedNAS: Federated deep learning via neural architecture search*. 2021. arXiv: 2004.08546 [cs.LG].

[11] D. Jin, N. Kannengießer, S. Rank, and A. Sunyaev. "Collaborative distributed machine learning." In: 57.4 (Dec. 2024). ISSN: 0360-0300. DOI: 10.1145/3704807.

[12] P. Kairouz, H. B. McMahan, B. Avent, A. Bellet, M. Bennis, A. N. Bhagoji, K. Bonawitz, Z. Charles, G. Cormode, R. Cummings, R. G. L. D'Oliveira, H. Eichner, S. E. Rouayheb, D. Evans, J. Gardner, Z. Garrett, A. Gascón, B. Ghazi, P. B. Gibbons, M. Gruteser, Z. Harchaoui, C. He, L. He, Z. Huo, B. Hutchinson, J. Hsu, M. Jaggi, T. Javidi, G. Joshi, M. Khodak, J. Konečný, A. Korolova, F. Koushanfar, S. Koyejo, T. Lepoint, Y. Liu, P. Mittal, M. Mohri, R. Nock, A. Özgür, R. Pagh, M. Raykova, H. Qi, D. Ramage, R. Raskar, D. Song, W. Song, S. U. Stich, Z. Sun, A. T. Suresh, F. Tramèr, P. Vepakomma, J. Wang, L. Xiong, Z. Xu, Q. Yang, F. X. Yu, H. Yu, and S. Zhao. *Advances and Open Problems in Federated Learning*. 2021. arXiv: 1912.04977 [cs.LG].

[13] S. Khan, A. Rizwan, A. N. Khan, M. Ali, R. Ahmed, and D. H. Kim. "A multi-perspective revisit to the optimization methods of Neural Architecture Search and Hyper-parameter optimization for non-federated and federated learning environments." In: *Computers and Electrical Engineering* 110 (2023), p. 108867. ISSN: 0045-7906. DOI: https://doi.org/10.1016/j.compeleceng.2023.108867.

[14] J. Liu, J. Yan, H. Xu, Z. Wang, J. Huang, and Y. Xu. "Finch: Enhancing federated learning with hierarchical neural architecture search." In: *IEEE Transactions on Mobile Computing* 23.5 (2024), pp. 6012–6026. DOI: 10.1109/TMC.2023.3315451.

[15] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas. "Communication-efficient learning of deep networks from decentralized data." In: *Proceedings of the 20th international conference on artificial intelligence and statistics*. Ed. by S. Aarti and Z. Jerry. Vol. 54. Proceedings of machine learning research. PMLR, Apr. 2017, pp. 1273–1282.

[16] M. J. Page, J. E. McKenzie, P. M. Bossuyt, I. Boutron, T. C. Hoffmann, C. D. Mulrow, L. Shamseer, J. M. Tetzlaff, E. A. Akl, S. E. Brennan, R. Chou, J. Glanville, J. M. Grimshaw, A. Hróbjartsson, M. M. Lalu, T. Li, E. W. Loder, E. Mayo-Wilson, S. McDonald, L. A. McGuinness, L. A. Stewart, J. Thomas, A. C. Tricco, V. A. Welch, P. Whiting, and D. Moher. "The PRISMA 2020 statement: an updated guideline for reporting systematic reviews." In: *BMJ* 372 (2021). DOI: 10.1136/bmj.n71. eprint: https://www.bmj.com/content/372/bmj.n71.full.pdf.

[17] R. E. Shawi. "AgingFedNAS: Aging evolution federated deep learning for architecture and hyperparameter search." In: *Advances in knowledge discovery and data mining*. Ed. by X. Wu, M. Spiliopoulou, C. Wang, V. Kumar, L. Cao, Y. Wu, Y. Yao, and Z. Wu. Singapore: Springer Nature Singapore, 2025, pp. 107–119. ISBN: 978-981-96-8173-0.

[18] A. Z. Tan, H. Yu, L. Cui, and Q. Yang. "Towards personalized federated learning." In: *IEEE Transactions on Neural Networks and Learning Systems* 34.12 (2023), pp. 9587–9603. DOI: 10.1109/TNNLS.2022.3160699.

[19] C. White, M. Safari, R. Sukthanker, B. Ru, T. Elsken, A. Zela, D. Dey, and F. Hutter. *Neural architecture search: Insights from 1000 papers*. 2023. arXiv: 2301.08727 [cs.LG].

[20] Y. Wu, H. Fan, W. Ying, Z. Zhou, Q. Zheng, and J. Zhang. "Federated Neural Architecture Search with Hierarchical Progressive Acceleration for Medical Image Segmentation." In: *Advances in Swarm Intelligence*. Ed. by Y. Tan and Y. Shi. Singapore: Springer Nature Singapore, 2024, pp. 112–123. ISBN: 978-981-97-7184-4.

[21] J. Yan, J. Liu, H. Xu, Z. Wang, and C. Qiao. "Peaches: Personalized federated learning with neural architecture search in edge computing." In: *IEEE Transactions on Mobile Computing* 23.11 (2024), pp. 10296–10312. DOI: 10.1109/TMC.2024.3373506.

[22] A. Yang and Y. Liu. "Heterogeneity-aware personalized federated neural architecture search." In: *Entropy* 27.7 (2025). ISSN: 1099-4300. DOI: 10.3390/e27070759.

[23] J. Yuan, M. Xu, Y. Zhao, K. Bian, G. Huang, X. Liu, and S. Wang. "Resource-Aware Federated Neural Architecture Search over Heterogeneous Mobile Devices." In: *IEEE Transactions on Big Data* (2022). DOI: 10.1109/TBDATA.2022.3227403.

[24] H. Zhu, H. Zhang, and Y. Jin. "From federated learning to federated neural architecture search: a survey." In: *Complex and Intelligent Systems* 7.2 (Apr. 2021), pp. 639–657. ISSN: 2198-6053. DOI: 10.1007/s40747-020-00247-z.

[25] B. Zoph and Q. V. Le. *Neural architecture search with reinforcement learning*. 2017. arXiv: 1611.01578 [cs.LG].